

Root Cause Analyzer

Incident Troubleshooting Chatbot Agent

Implementation Plan | 26 May 2026 | v1.0

Python 3.x

LangChain

ChromaDB

OpenAI (Model-Agnostic)

FastAPI

On-Premises

Executive Summary

This document outlines a 6-phase implementation plan for the Root Cause Analyzer — an AI-powered chatbot that helps support engineers analyze and troubleshoot incidents. The system combines a RAG pipeline over Datapower/ESGA runbooks with optional Splunk log analysis to deliver grounded, cited recommendations through a multi-turn conversational interface.

Runbooks	20–30 Word documents (free-form prose), ingested once manually
Logs	Optional Datapower JSON logs (20–30 lines), summarized before use
Conversation	Multi-turn, stateless — history managed client-side
LLM	OpenAI (Phase 1); model-agnostic abstraction for future providers
Retrieval	HyDE + Hybrid (Dense + BM25) + Reciprocal Rank Fusion
Deployment	On-premises app server; LLM via enterprise cloud API gateway

System Architecture

```
Web Portal (conversation history lives client-side)
| REST API – each request carries full history
FastAPI Backend (On-Premises)
|
+---+-----+-----+-----+
| | |
Log Summarizer RAG Pipeline LLM Abstraction Layer
| (LangChain) (OpenAI / Anthropic / ...)
| | |
+-----+ ChromaDB + BM25 Enterprise LLM API
| (On-Premises) (Cloud)
|
Ingestion Pipeline (manual CLI trigger)
Word Docs → Parser → Chunker → Embedder → ChromaDB
```

Phase Summary

Phase	Focus	Key Output
1	Ingestion Pipeline & Knowledge Base	Runbooks in ChromaDB + BM25 index
2	Core RAG Pipeline (Single Turn)	Working /analyze API with citations
3	Advanced RAG	HyDE + Hybrid Retrieval + RRF
4	Splunk Log Integration	Log summarizer, enriched context
5	Multi-turn Stateless Conversation	Full conversation support
6	Hardening & Production Readiness	Model-agnostic, prod-ready app

PHASE 1**Ingestion Pipeline & Knowledge Base**

Goal: Parse Word doc runbooks, chunk intelligently, embed and store in ChromaDB.

Components

Document Parser — python-docx extracts text preserving heading hierarchy and tables.

Section-aware Chunker — Chunk by heading structure — NOT naive token split. Symptoms, resolution steps and error codes stay together. Each chunk tagged with metadata: runbook_name, section_title, pattern_id, chunk_index.

Embedding — OpenAI text-embedding-3-small, abstracted behind the LLM layer.

ChromaDB — Fresh local install, one collection for runbooks.

BM25 Index — Built in parallel alongside ChromaDB using rank_bm25, serialized to disk. Required for hybrid retrieval in Phase 3.

CLI Trigger — python ingest.py --docs ./runbooks/ — manual execution only.

Deliverables

- ✓ Ingestion script (ingest.py)
- ✓ ChromaDB populated with all 20-30 runbooks
- ✓ BM25 index persisted to disk
- ✓ Metadata schema defined and documented

PHASE 2 Core RAG Pipeline (Single Turn)

Goal: Retrieve relevant runbook chunks and generate a grounded recommendation with citations.

Components

LLM Abstraction Layer — Thin wrapper using LangChain's BaseChatModel interface. OpenAI is the first implementation. Swapping to Anthropic requires only a config change — no pipeline changes.

Basic Retriever — ChromaDB dense vector search (top-k). Advanced retrieval replaces this in Phase 3.

Prompt Template — Structured system prompt: answer only from retrieved context, cite runbook name and section, acknowledge explicitly if no relevant runbook found.

FastAPI Endpoint — POST /api/analyze accepts { incident: str, history: [] }, returns { answer: str, sources: [] }.

Source Citation — Each retrieved chunk carries metadata through to the response, formatted as: [Runbook: Pattern_7, Section: 5.1 Certificate Validation].

Deliverables

- ✓ Working end-to-end RAG for single-turn incident queries
- ✓ FastAPI app skeleton with /analyze endpoint
- ✓ LLM abstraction layer (model-agnostic from day one)
- ✓ Versioned prompt templates

PHASE 3**Advanced RAG — HyDE + Hybrid Retrieval + RRF**

Goal: Improve retrieval quality using techniques suited to free-form prose runbooks.

Components

HyDE (Hypothetical Document Embeddings) — Before retrieval, the LLM generates a hypothetical runbook passage that would answer the incident query. That passage is embedded and used as the retrieval query. Dramatically improves recall when users don't use exact runbook terminology.

Hybrid Retrieval — Embedded HyDE query runs against ChromaDB (dense) AND BM25 index (sparse) in parallel. BM25 catches exact error codes (0x80090326, field names) that semantic search misses.

Reciprocal Rank Fusion (RRF) — Merges the dense and sparse result lists into a single ranked list. Top-k chunks from merged list go to the LLM.

Optional Reranker — Cross-encoder reranker as a flag-gated optional step for further refinement.

Retrieval Flow

```
Incident Description
→ HyDE: LLM generates hypothetical runbook answer
→ [ChromaDB dense search] + [BM25 sparse search]
→ RRF merge → Top-K chunks → LLM
```

Deliverables

- ✓ HyDE query expansion module
- ✓ Hybrid retriever (replaces Phase 2 basic retriever)
- ✓ RRF scoring utility
- ✓ Retrieval quality measurable against test incidents

PHASE 4 Splunk Log Integration

Goal: Allow users to optionally paste Datapower JSON logs. Agent summarizes them and enriches context.

Components

Log Parser — Accepts raw Datapower JSON logs (20-30 lines). Extracts key fields: timestamp, error_code, transaction_id, src_ip, uri, http_status, dp_error_message.

Log Summarizer — Dedicated LLM call before the main RAG call. Prompt: summarize key errors, patterns and anomalies in 5-8 bullet points. Summary (not raw logs) enters the context.

Context Enrichment — Log summary injected into the retrieval query and final LLM prompt as a clearly labeled separate block. Keeps the main RAG prompt clean, avoids token bloat.

Optional Handling — If no logs provided, pipeline skips this step entirely. No impact on normal flow.

API Update — POST /api/analyze updated to accept { incident: str, logs: str|null, history: [] }.

Prompt Structure

```
[System] You are an incident troubleshooting assistant...
[Runbook Context]
[Log Summary] ← injected here
[History]
[User]
```

Deliverables

- ✓ Log parser + summarizer module
- ✓ Updated prompt template with log summary block
- ✓ Updated API contract (logs field optional)

PHASE 5 Multi-turn Stateless Conversation

Goal: Support follow-up questions within a session without any server-side state storage.

Components

Stateless Design — Server never stores conversation history. Frontend passes the full history array on every request as a list of {role, content} pairs.

History Window Management — Server trims history to last N turns (configurable, default 10) to prevent context window overflow before sending to LLM.

LangChain ConversationChain — History is reconstructed from the passed array each request and fed into LangChain's message format.

Context-aware Retrieval — For follow-up queries (e.g. 'what about the timeout issue?'), the retrieval query combines the latest user message with a brief summary of recent history so retrieval remains relevant.

Deliverables

- ✓ Stateless multi-turn conversation support
- ✓ Configurable history window trimming
- ✓ Context-aware retrieval for follow-up questions

PHASE 6**Hardening & Production Readiness**

Goal: Make the system reliable, observable, maintainable and model-agnostic.

Components

Model-agnostic Config — MODEL_PROVIDER=openai|anthropic|azure_openai environment variable. LLM abstraction layer routes accordingly. Adding a new provider = one new class, zero pipeline changes.

Error Handling — Graceful degradation if retrieval returns no relevant chunks — agent states this explicitly, never hallucinates content.

Prompt Versioning — Prompts stored as versioned template files, not hardcoded in source.

Logging & Tracing — Structured logging per request: retrieval scores, tokens used, latency per step. No conversation content logged (privacy).

Health Check — GET /health endpoint for infrastructure monitoring.

Re-ingestion Script — Clean wipe + re-ingest command for runbook updates.

Deliverables

- ✓ Production-ready FastAPI application
- ✓ Environment-based model provider config
- ✓ Structured logging (no PII)
- ✓ Docker setup + on-premises deployment guide
- ✓ Re-ingestion utility script

Appendix — Key Design Decisions

Decision	Choice	Rationale
Chunking strategy	Section-aware (by heading)	Keeps related content (symptoms, steps, codes) together
Retrieval method	Hybrid: Dense + BM25 + RRF	Dense misses exact error codes; BM25 misses semantic intent
Query expansion	HyDE	Users describe symptoms, not runbook terminology
Log handling	Summarize before context	20-30 lines JSON → 5-8 bullets; avoids token waste
Session state	Stateless (client-side history)	No server storage needed; simpler infra
LLM abstraction	LangChain BaseChatModel	One config change to swap provider, no pipeline changes
Ingestion trigger	Manual CLI	Runbooks change rarely; automation not justified
Vector store	ChromaDB (fresh, on-prem)	Lightweight, no external dependencies, fits scale